MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Operating Systems
Lab 1: The OS as a Resource Manager
Lab 2: Meet the OS You Will Build (xv6)

Checked by:
univ. assist. / lect. Patricia Reițman
FCIM, UTM

Chisinau – 2026

Contents

1 LAB 1: THE OS AS A RESOURCE MANAGER **3**

1.1 Part 1: Files and directories 3

1.2 Part 2: Processes 5

1.3 Part 3: Memory 7

1.4 Part 4: Devices and storage 7

2 LAB 2: MEET THE OS YOU WILL BUILD (XV6) **10**

2.1 Part 2: Use xv6 as the Unix it is 10

2.2 Part 3: Read the source 10

CONCLUSION **13**

1 LAB 1: THE OS AS A RESOURCE MANAGER

1.1 Part 1: Files and directories

- **Ownership:** The files are owned by the user **vitalie** and belong to the group **vitalie**[cite: 3].
- **Permissions:** The characters `-rw-------` mean `note.txt` is a regular file (`-`) where the owner has read and write permissions (`rw-`), while the group (`---`) and all other users (`---`) have absolutely no access[cite: 3].
- **Root Directories:** Based on the `ls -la /` output, two examples of root directories are[cite: 3]:
  - **`/etc`**: Holds system-wide configuration files[cite: 3].
  - **`/home`**: Contains the personal directories for standard users (like the `vitalie` folder)[cite: 3].
  - **`/dev`**: Holds device files that interface with hardware (or virtualized hardware in WSL)[cite: 3].

Terminal Output:

```
vitalie@Lenovo:/mnt/c/Users/vital$ mkdir -p ~/os-lab1 && cd ~/os-lab1
whoami
uname -a
uptime
vitalie
Linux Lenovo 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5
    18:31:42 UTC 2025 aarch64 aarch64 aarch64 GNU/Linux
 17:42:20 up 5 min,  1 user,  load average: 0.62, 0.26, 0.09
vitalie@Lenovo:~/os-lab1$ pwd
ls -la /
ls -la ~
cd /etc && ls | head
/home/vitalie/os-lab1
total 2748
drwxr-xr-x  33 root root    4096 Sep 28 17:37 .
drwxr-xr-x  33 root root    4096 Sep 28 17:37 ..
drwxr-xr-x   3 root root    4096 Sep 19  2025 Docker
lrwxrwxrwx   1 root root       7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x   2 root root    4096 Feb 26  2024 bin.usr-is-merged
drwxr-xr-x   2 root root    4096 Apr 22  2024 boot
drwxr-xr-x  16 root root    4080 Sep 28 17:37 dev
drwxr-xr-x  93 root root    4096 Sep 28 17:41 etc
drwxr-xr-x   3 root root    4096 Sep 19  2025 home
-rwxrwxrwx   1 root root 2681288 Aug  6  2025 init
lrwxrwxrwx   1 root root       7 Apr 22  2024 lib -> usr/lib
drwxr-xr-x   2 root root    4096 Apr  8  2024 lib.usr-is-merged
drwx------   2 root root   16384 Sep 19  2025 lost+found
drwxr-xr-x   2 root root    4096 Aug  5  2025 media
drwxr-xr-x   5 root root    4096 Sep 19  2025 mnt
drwxr-xr-x   2 root root    4096 Aug  5  2025 opt
dr-xr-xr-x 253 root root       0 Sep 28 17:37 proc
drwx------   5 root root    4096 Sep 19  2025 root
```

```
drwxr-xr-x  21 root root     620 Sep 28 17:41 run
lrwxrwxrwx   1 root root       8 Apr 22  2024 sbin -> usr/sbin
drwxr-xr-x   2 root root    4096 Mar 31  2024 sbin.usr-is-merged
drwxr-xr-x   2 root root    4096 Sep 19  2025 snap
drwxr-xr-x   2 root root    4096 Aug  5  2025 srv
dr-xr-xr-x  13 root root       0 Sep 28 17:36 sys
drwxrwxrwt   9 root root    4096 Sep 28 17:41 tmp
drwxr-xr-x  12 root root    4096 Sep 28 17:41 usr
drwxr-xr-x  13 root root    4096 Sep 19  2025 var
drwx------   2 root root    4096 Sep 28 17:29 wslAHkCgn
drwx------   2 root root    4096 Sep 28 17:36 wslCEdfNc
drwx------   2 root root    4096 Sep 28 17:36 wslCPLPAa
drwx------   2 root root    4096 Sep 28 17:29 wslELpgfp
drwx------   2 root root    4096 Sep 28 17:36 wslGoHKNc
drwx------   2 root root    4096 Sep 28 17:29 wslIfOKfp
drwx------   2 root root    4096 Sep 28 17:29 wslaJBPdp
drwx------   2 root root    4096 Sep 28 17:36 wslgOFmIc
drwx------   2 root root    4096 Sep 28 17:29 wsliljpfp
drwx------   2 root root    4096 Sep 28 17:36 wsloDbbMc
total 36
drwxr-x--- 5 vitalie vitalie 4096 Sep 28 17:42 .
drwxr-xr-x 3 root    root    4096 Sep 19  2025 ..
lrwxrwxrwx 1 vitalie vitalie   23 Sep 19  2025 .aws -> /mnt/c/Users/vital/.aws
lrwxrwxrwx 1 vitalie vitalie   25 Sep 19  2025 .azure -> /mnt/c/Users/vital/.azure
-rw------- 1 vitalie vitalie  112 Sep 19  2025 .bash_history
-rw-r--r-- 1 vitalie vitalie  220 Sep 19  2025 .bash_logout
-rw-r--r-- 1 vitalie vitalie 3771 Sep 19  2025 .bashrc
drwx------ 2 vitalie vitalie 4096 Sep 19  2025 .cache
drwxr-xr-x 4 vitalie vitalie 4096 Sep 19  2025 .docker
-rw-rw-r-- 1 vitalie vitalie    0 Sep 28 17:37 .motd_shown
-rw-r--r-- 1 vitalie vitalie  807 Sep 19  2025 .profile
-rw-r--r-- 1 vitalie vitalie    0 Sep 28 17:38 .sudo_as_admin_successful
drwxr-xr-x 2 vitalie vitalie 4096 Sep 28 17:42 os-lab1
PackageKit
X11
adduser.conf
alternatives
apparmor
apparmor.d
apport
apt
bash.bashrc
bash_completion
vitalie@Lenovo:/etc$ cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
total 8
-rw-r--r-- 1 vitalie vitalie 24 Sep 28 17:42 note.txt
-rw-r--r-- 1 vitalie vitalie 24 Sep 28 17:42 renamed.txt
vitalie@Lenovo:~/os-lab1/demo$ ls -l note.txt
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
-rw-r--r-- 1 vitalie vitalie 24 Sep 28 17:42 note.txt
```

```
-rw------- 1 vitalie vitalie 24 Sep 28 17:42 note.txt
```

1.2 Part 2: Processes

- **Process State:** The "State:" line in `/proc/3929/status` says `S (sleeping)`[cite: 3].

Terminal Output:

```
vitalie@Lenovo:~/os-lab1/demo$ ps aux | head
ps aux | wc -l
top
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.3  0.1  22060 12544 ?        Ss   17:36   0:01 /usr/lib/systemd/
   systemd --system --deserialize=53
root           2  0.0  0.0   3008  1920 ?        Sl   17:36   0:00 /init
root           6  0.0  0.0   3008  1920 ?        Sl   17:36   0:00 plan9 --control-
   socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root          42  0.0  0.2  66676 17964 ?        S<s  17:36   0:00 /usr/lib/systemd/
   systemd-journald
root          87  0.0  0.0  26312  6784 ?        Ss   17:36   0:00 /usr/lib/systemd/
   systemd-udevd
systemd+     140  0.0  0.1  21364 12160 ?        Ss   17:36   0:00 /usr/lib/systemd/
   systemd-resolved
systemd+     145  0.0  0.0  90760  6784 ?        Ssl  17:36   0:00 /usr/lib/systemd/
   systemd-timesyncd
root         158  0.0  0.0   4160  2176 ?        Ss   17:36   0:00 /usr/sbin/cron -f
   -P
message+     162  0.0  0.0   9660  4224 ?        Ss   17:36   0:00 @dbus-daemon --
   system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
27
top - 17:45:19 up 8 min,  1 user,  load average: 0.03, 0.14, 0.07
Tasks:  25 total,   1 running,  24 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7726.5 total,   5614.0 free,    523.8 used,   1771.8 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7202.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   22060  12544   8832 S   0.0   0.2   0:01.84 systemd
      2 root      20   0    3008   1920   1920 S   0.0   0.0   0:00.01 init-systemd(
        Ub
      6 root      20   0    3008   1920   1920 S   0.0   0.0   0:00.00 init
     42 root      19  -1   66676  17964  17068 S   0.0   0.2   0:00.30 systemd-
        journal
     87 root      20   0   26312   6784   4480 S   0.0   0.1   0:00.20 systemd-udevd
    140 systemd+  20   0   21364  12160   9984 S   0.0   0.2   0:00.10 systemd-
        resolve
    145 systemd+  20   0   90760   6784   6016 S   0.0   0.1   0:00.05 systemd-
        timesyn
    158 root      20   0    4160   2176   2048 S   0.0   0.0   0:00.00 cron
    162 message+  20   0    9660   4224   3712 S   0.0   0.1   0:00.19 dbus-daemon
    175 root      20   0   17720   7552   6656 S   0.0   0.1   0:00.17 systemd-logind
    179 root      20   0 1755592  11776   9728 S   0.0   0.1   0:00.13 wsl-pro-
        service
    189 syslog    20   0  222812   4608   3584 S   0.0   0.1   0:00.07 rsyslogd
    199 root      20   0    2732   1536   1536 S   0.0   0.0   0:00.00 agetty
    202 root      20   0    2688   1536   1536 S   0.0   0.0   0:00.00 agetty
```

```
    211 root      20   0  107904  21888  12544 S   0.0   0.3   0:00.11 unattended-
       upgr
    298 root      20   0    8416   3328   2944 S   0.0   0.0   0:00.00 login
    344 vitalie   20   0   20208  10752   8704 S   0.0   0.1   0:00.12 systemd
    345 vitalie   20   0   22192   3256   1792 S   0.0   0.0   0:00.00 (sd-pam)
    372 vitalie   20   0    5764   4736   3200 S   0.0   0.1   0:00.01 bash
    450 root      20   0    3016   1036    896 S   0.0   0.0   0:00.00 SessionLeader
    451 root      20   0    3032    916    768 S   0.0   0.0   0:00.10 Relay(452)
    452 vitalie   20   0    5776   4864   3200 S   0.0   0.1   0:00.07 bash
    916 root      20   0  373944  19328  16512 S   0.0   0.2   0:00.04 packagekitd
    921 polkitd   20   0  309068   7296   6528 S   0.0   0.1   0:00.05 polkitd
   3925 vitalie   20   0    9236   5120   3072 R   0.0   0.1   0:00.00 top

vitalie@Lenovo:~/os-lab1/demo$ sleep 300 &
jobs
ps -ef | grep sleep
kill <PID>
[1] 3926
[1]+  Running                 sleep 300 &
vitalie     3926     452  0 17:45 pts/0    00:00:00 sleep 300
vitalie     3928     452  0 17:45 pts/0    00:00:00 grep --color=auto sleep
-bash: syntax error near unexpected token `newline'
vitalie@Lenovo:~/os-lab1/demo$ sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20
kill $PID
[2] 3929
attr        comm             fd                latency    mountinfo   oom_adj
          root       smaps_rollup  task
autogroup   coredump_filter  fdinfo            limits     mounts      oom_score
         sched      stack         timens_offsets
auxv        cpuset           gid_map           loginuid   mountstats  oom_score_adj
    schedstat  stat          timers
cgroup      cwd              io                map_files  net         pagemap
          sessionid  statm         timerslack_ns
clear_refs  environ          ksm_merging_pages  maps       ns          personality
       setgroups  status        uid_map
cmdline     exe              ksm_stat          mem        numa_maps   projid_map
       smaps      syscall       wchan
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   3929
Ngid:   0
Pid:    3929
PPid:   452
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 1000 1001
NStgid: 3929
NSpid:  3929
NSpgid: 3929
NSsid:  452
Kthread:        0
VmPeak:     2700 kB
```

```
VmSize:     2700 kB
VmLck:         0 kB
vitalie@Lenovo:~/os-lab1/demo$
```

1.3 Part 3: Memory

- **(1) Total and free RAM:** The VM has 7.5 GiB of total RAM, and 7.1 GiB is currently free[cite: 3].
- **(2) Swap space:** Swap is a space on the hard drive used as virtual memory when physical RAM runs out[cite: 3]. The system has 2.0 GiB of swap configured[cite: 3].
- **(3) VmRSS of `sleep`:** The bare `sleep` process uses 1536 kB of resident memory[cite: 3]. This might seem surprisingly large for a program that literally does nothing, but it needs that space to load its basic executable structure and the standard C libraries[cite: 3].

Terminal Output:

```
[2]+  Terminated              sleep 300
vitalie@Lenovo:~/os-lab1/demo$ free -h
cat /proc/meminfo | head -6
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
               total        used        free      shared  buff/cache   available
Mem:           7.5Gi       489Mi       7.1Gi       3.5Mi       144Mi       7.1Gi
Swap:          2.0Gi          0B       2.0Gi
MemTotal:        7911940 kB
MemFree:         7422036 kB
MemAvailable:    7409792 kB
Buffers:            1076 kB
Cached:            97192 kB
SwapCached:            0 kB
[2] 3944
VmRSS:      1536 kB
vitalie@Lenovo:~/os-lab1/demo$
```

1.4 Part 4: Devices and storage

- **(1) Root filesystem device:** The root filesystem (`/`) is mounted on the device `/dev/sdd` (as shown in both the `df -h` and `lsblk` outputs)[cite: 3].
- **(2) `/dev` entry:** For a `/dev` entry, `console` from the `ls -l /dev` output stands for the physical (or virtual) terminal console interface[cite: 3]. Alternatively, `sdc` (from the `lsblk` output) stands for a virtual hard disk being used as the SWAP space[cite: 3].
- **(3) "Everything is a file":** Based on the lab text, "Everything is a file" means the OS hides every disk, terminal, and device behind uniform file-like interfaces[cite: 3]. This was seen firsthand when running `ls -l` on devices in the `/dev` folder exactly the same way regular text files are listed[cite: 3].

Terminal Output:

```
[2]+  Terminated              sleep 300
vitalie@Lenovo:~/os-lab1/demo$ df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
Filesystem      Size  Used Avail Use% Mounted on
none            3.8G     0  3.8G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            3.8G  4.0K  3.8G   1% /mnt/wsl
drivers         476G  265G  212G  56% /usr/lib/wsl/drivers
/dev/sdd       1007G  2.7G  953G   1% /
none            3.8G   80K  3.8G   1% /mnt/wslg
none            3.8G     0  3.8G   0% /usr/lib/wsl/lib
rootfs          3.8G  2.6M  3.8G   1% /init
none            3.8G  572K  3.8G   1% /run
none            3.8G     0  3.8G   0% /run/lock
none            3.8G     0  3.8G   0% /run/shm
none            3.8G   76K  3.8G   1% /mnt/wslg/versions.txt
none            3.8G   76K  3.8G   1% /mnt/wslg/doc
C:\             476G  265G  212G  56% /mnt/c
tmpfs           773M   20K  773M   1% /run/user/1000
tmpfs           3.8G     0  3.8G   0% /run/qemu
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 369.9M  1 disk
sdb   8:16   0 286.4M  1 disk
sdc   8:32   0     2G  0 disk [SWAP]
sdd   8:48   0     1T  0 disk /mnt/wslg/distro
                               /
12K     /home/vitalie/os-lab1
total 0
crw-r--r-- 1 root root     10, 235 Sep 28 17:37 autofs
drwxr-xr-x 2 root root         280 Sep 28 17:37 block
drwxr-xr-x 2 root root         120 Sep 28 17:36 bsg
crw------- 1 root root     10, 234 Sep 28 17:37 btrfs-control
drwxr-xr-x 2 root root        3380 Sep 28 17:37 char
crw------- 1 root root      5,   1 Sep 28 17:41 console
lrwxrwxrwx 1 root root          11 Sep 28 17:37 core -> /proc/kcore
crw------- 1 root root     10, 122 Sep 28 17:37 cpu_dma_latency
crw------- 1 root root     10, 203 Sep 28 17:37 cuse
none on /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2 type overlay (rw,nosuid,nodev,noatime,lowerdir=/modules,upperdir=/lib/modules/6.6.87.2-microsoft-standard-WSL2/rw/upper,workdir=/lib/modules/6.6.87.2-microsoft-standard-WSL2/rw/work,uuid=on)
none on /mnt/wsl type tmpfs (rw,relatime)
drivers on /usr/lib/wsl/drivers type 9p (ro,nosuid,nodev,noatime,aname=drivers;fmask=222;dmask=222,cache=5,access=client,msize=65536,trans=fd,rfd=8,wfd=8)
/dev/sdd on / type ext4 (rw,relatime,discard,errors=remount-ro,data=ordered)
none on /mnt/wslg type tmpfs (rw,relatime)
/dev/sdd on /mnt/wslg/distro type ext4 (ro,relatime,discard,errors=remount-ro,data=ordered)
none on /usr/lib/wsl/lib type overlay (rw,nosuid,nodev,noatime,lowerdir=/gpu_lib_packaged:/gpu_lib_inbox,upperdir=/gpu_lib/rw/upper,workdir=/gpu_lib/rw/work,uuid=on)
rootfs on /init type rootfs (ro,size=3941700k,nr_inodes=985425)
none on /dev type devtmpfs (rw,nosuid,relatime,size=3941700k,nr_inodes=985425,mode
```

```
    =755)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,noatime)
vitalie@Lenovo:~/os-lab1/demo$
```

2 LAB 2: MEET THE OS YOU WILL BUILD (XV6)

2.1 Part 2: Use xv6 as the Unix it is

- **(1) Three programs xv6 ships with:** Three programs xv6 ships with are `ls`, `cat`, and `echo` (along with `grep`, `wc`, or `sh`)[cite: 3].

- **(2) Pipe OS features:** For a pipe (`|`) to work, the OS must support **processes** (since a pipe sends the output of one process as the input to another) and **communication** (Inter-Process Communication, or IPC, to handle the data transfer)[cite: 3].

- **(3) Shell comparison:** The xv6 shell acts and feels exactly like the Linux shell from Lab 1 because it is built on the same fundamental Unix design principles (the ancestor of Linux)[cite: 3].

2.2 Part 3: Read the source

- **(1) `cat.c` system calls:** The `user/cat.c` program uses `read()`, `write()`, and `exit()`[cite: 3]. `read()` asks the kernel to pull data from a file descriptor into a buffer, `write()` asks the kernel to output that data to the screen (or another file), and `exit()` tells the kernel to terminate the program[cite: 3].

- **(2) `sys_read` location:** As shown by the `grep` output, `sys_read` is implemented in the `kernel/sysfile.c` file on line 69[cite: 3].

- **(3) `kernel/` vs `user/`:** The `kernel/` folder contains the operating system itself which runs privileged, whereas `user/` holds the ordinary programs that just run on top of it and make requests to the kernel[cite: 3].

Terminal Output:

```
vitalie@Lenovo:~/xv6-riscv$ sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;
```

```
21
22    // Return blank-padded name.
23    if (strlen(p) >= DIRSIZ)
24      return p;
25    memmove(buf, p, strlen(p));
26    memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
27    buf[sizeof(buf) - 1] = '\0';
28    return buf;
29  }
30
31  void
32  ls(char *path)
33  {
34    char buf[512], *p;
35    int fd;
36    struct dirent de;
37    struct stat st;
38
39    if ((fd = open(path, O_RDONLY)) < 0) {
40      fprintf(2, "ls: cannot open %s\n", path);
41      return;
42    }
43
44    if (fstat(fd, &st) < 0) {
45  #include "kernel/types.h"
46  #include "kernel/fcntl.h"
47  #include "user/user.h"
48
49  char buf[512];
50
51  void
52  cat(int fd)
53  {
54    int n;
55
56    while ((n = read(fd, buf, sizeof(buf))) > 0) {
57      if (write(1, buf, n) != n) {
58        fprintf(2, "cat: write error\n");
59        exit(1);
60      }
61    }
62    if (n < 0) {
63      fprintf(2, "cat: read error\n");
64      exit(1);
65    }
66  }
67
68  int
69  main(int argc, char *argv[])
70  {
71    int fd, i;
72
73    if (argc <= 1) {
74      cat(0);
75  #define SBRK_ERROR ((char *)-1)
76
77  struct stat;
78
```

```
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);
69:sys_read(void)
83:sys_write(void)
```

CONCLUSION

This practical work provided a hands-on understanding of how an operating system functions as a resource manager and how kernel space differs from user space. By interacting directly with the Linux environment and compiling the xv6 operating system, theoretical concepts surrounding process management, memory allocation, and file systems were translated into observable terminal commands and C code.

Key takeaways from these labs include identifying hardware devices presented as files in the `/dev` directory, managing background processes via the `/proc` pseudo-filesystem, and recognizing the critical boundary between user-space system calls and privileged kernel implementations. The debugging required to compile the OS effectively demonstrated the rigid demands of operating system architecture.